	BTS SIO **Services Informatiques aux Organisations**		
	Option	SLAM	
	Session	**2025/26**	

Nom et prénom : Costantini Yaacov-Haï	**Activité professionnelle N° :**	**6**

Intitulé de l'activité	**Développement procédural d'un chat instantané – Evolution V1 vers V2 (PHP, MySQL, Bootstrap)**
Contexte	Dans le cadre de mon projet personnel de messagerie instantanée, j'ai développé une première version fonctionnelle du chat en PHP procédural avec une base de données MySQL. Le projet a évolué en deux itérations successives : une ChatV1 centrée sur les fonctionnalités de base (envoi et affichage de messages), puis une ChatV2 intégrant la gestion des utilisateurs (inscription et connexion) et une interface améliorée avec Bootstrap. Cette approche itérative m'a permis de construire progressivement l'application en consolidant les acquis à chaque étape. La ChatV3, quant à elle, fera l'objet d'une activité distincte portant sur la migration vers une architecture MVC.
Objectifs	• Développer une application de chat fonctionnelle en PHP procédural sans framework backend • Mettre en place une base de données MySQL pour stocker les utilisateurs et les messages • Faire évoluer l'application de manière itérative : fonctionnalités de base (V1) puis gestion des utilisateurs et amélioration de l'interface (V2) • Utiliser Bootstrap pour obtenir une interface responsive et soignée
Lieu de réalisation	CFA OZAR

SOLUTIONS ENVISAGEABLES

Plusieurs approches auraient pu être envisagées pour développer cette application :

1. Utiliser un framework PHP (Laravel, Symfony) qui aurait accéléré le développement grâce à ses composants intégrés. Solution écartée car elle masque les mécanismes fondamentaux du PHP et n'est pas adaptée à un apprentissage progressif.
2. Utiliser un framework JavaScript full-stack (Node.js + Express) avec gestion native des WebSockets pour le temps réel. Solution plus moderne mais hors du périmètre des compétences visées à ce stade.
3. Développement en PHP procédural pur avec mysqli et MySQL, sans framework, permettant de maîtriser pleinement chaque ligne de code et de comprendre les bases du développement web dynamique. Solution retenue car elle correspond aux objectifs pédagogiques et au niveau BTS SIO

DESCRIPTION DE LA SOLUTION RETENUE	
Conditions initiales	Aucune application existante. Le développement part de zéro, en environnement local, avec PHP, un serveur Apache (XAMPP ou équivalent) et une base de données MySQL.
Conditions finales	À l'issue de la V2, l'application permet à un utilisateur de s'inscrire, de se connecter et d'échanger des messages avec d'autres utilisateurs connectés. L'interface est mise en forme avec Bootstrap et CSS personnalisé. La base de données MySQL gère les tables users et messages via mysqli en mode procédural.
Outils utilisés	PHP (mysqli), MySQL, HTML / CSS, Bootstrap, Serveur local Apache (Wamp), Navigateur web (Tests)

CONDITIONS DE REALISATION	
Matériels	Poste de travail personnel, environnement de développement local
Logiciels	Wamp server, Git, VSCode
Durée	4H
Contraintes	• Ne pas utiliser de framework backend • Rester en PHP procédural (pas d'orienté objet) • Travailler exclusivement en local • Assurer la continuité fonctionnelle entre chaque version

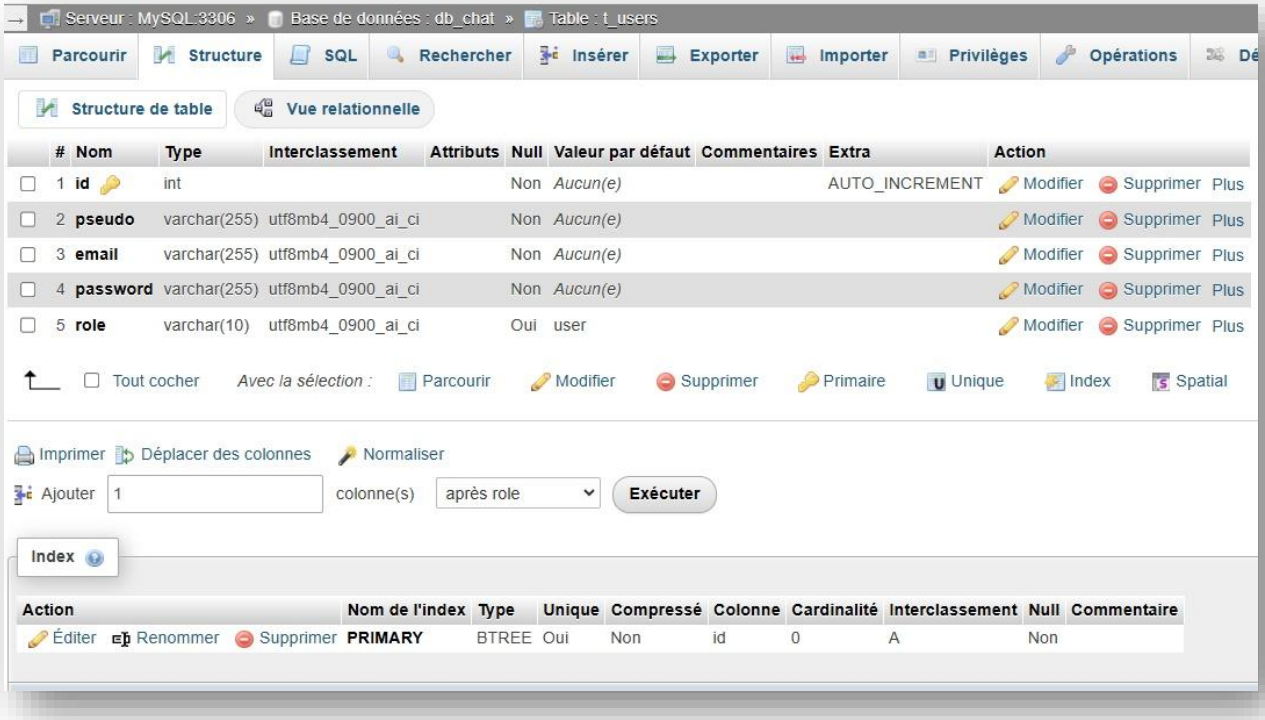
COMPETENCES MISES EN OEUVRE POUR CETTE ACTIVITE PROFESSIONNELLE	
Code	Intitulé
B2.1	Développer une application
B2.1	Concevoir des interfaces utilisateur
B2.3	Concevoir une base de données
B2.3	Exploiter les ressources du système d'information

DEROULEMENT DE L'ACTIVITE
Etapes de réalisations : 1. Création de la BDD **2.** ChatV1 – Fonctionnalités de base **3.** ChatV2 – Gestion des utilisateurs et amélioration de l’interface

I. Création de la BDD

Le chat instantané fonctionne grâce à une base de données dans laquelle sont enregistrés les messages. Pour vulgariser, nous pouvons dire qu'elle se résume à 2 simples tables en relation : la table *users* et la table *messages*. La relation entre les 2 tables est l'identifiant (id) utilisateur qui sera utilisé 2 fois : une fois pour la colonne *expéditeur* et une fois pour la colonne *destinataire*. Ces 2 colonnes seront des clés étrangères (Contraintes) dans la table *messages* associées à l'*id* (clé primaire) de la table *users.* Le destinataire sera sélectionné par l'utilisateur grâce à une liste déroulante affichant les noms des contacts (les *users*) et ayant comme valeur leur *id.* La partie conception et modélisation pourraient faire l'objet d'une documentation spécifique où il y sera éventuellement précisé comment l'importer sur un serveur base de données avec l'utilisation de PostgreSQL. Néanmoins, voici à quoi ressemble la BDD sur *phpmyadmin* avec Wamp :

A) Structure de la table *t_users* :



B) Contenu de la table *t_users :*

```
SELECT * FROM `t_users`
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Cr

Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par

Options supplémentaires

←T→	id	pseudo	email	password	role
Éditer Copier Supprimer	1	Yaacov	yaacov.costantini@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	4	Rephael	rephael.athlan@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	5	Avi Sabban	avi.sabban@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	6	Ethan	ethan.nadjar@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	7	Jonathan	jonathan.gabison@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	8	Nathanel	nathanel.bouhsira@ozar.fr	P@$$w0rd	user
Éditer Copier Supprimer	9	Fred	fred.guez@ozar.fr	P@$$w0rd	user

C) Structure de la table *t_messages* avec les 2 contraintes *:*

Serveur : MySQL:3306 » Base de données : db_chat » Table : t_messages

Parcourir | Structure | SQL | Rechercher | Insérer | Exporter | Importer | Privilèges | Opérations | Déc

Structure de table | Vue relationnelle

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	**id**	int			Non	*Aucun(e)*		AUTO_INCREMENT	Modifier Supprimer Plus
2	**expediteur**	int			Non	*Aucun(e)*			Modifier Supprimer Plus
3	**message**	longtext	utf8mb4_0900_ai_ci		Non	*Aucun(e)*			Modifier Supprimer Plus
4	**date**	time			Oui	*NULL*			Modifier Supprimer Plus
5	**destinataire**	int			Non	*Aucun(e)*			Modifier Supprimer Plus

Tout cocher *Avec la sélection :* Parcourir Modifier Supprimer Primaire Unique Index Spatial

Imprimer Déplacer des colonnes Normaliser

Ajouter 1 colonne(s) après destinataire Exécuter

Index

Action	Nom de l'index	Type	Unique	Compressé	Colonne	Cardinalité	Interclassement	Null	Commentaire
Éditer Renommer Supprimer	**PRIMARY**	BTREE	Oui	Non	id	20	A	Non	
Éditer Renommer Supprimer	**fk_id_dest**	BTREE	Non	Non	destinataire		A	Non	
Éditer Renommer Supprimer	**fk_id_expe**	BTREE	Non	Non	expediteur	0	A	Non	

D) Contenu de la table *t_messages* :

```
SELECT * FROM `t_messages`
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PH

Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Options supplémentaires

←T→				id	expediteur	message	date	destinataire
Éditer	Copier	Supprimer		8	1	coucou	22:06:13	4
Éditer	Copier	Supprimer		11	1	coucou	22:11:58	5
Éditer	Copier	Supprimer		12	5	Salut, c'est Avi !	22:36:41	1
Éditer	Copier	Supprimer		13	5	Bonjour mon cher !	23:01:52	6
Éditer	Copier	Supprimer		14	6	Coucou mon ami !	23:04:41	5
Éditer	Copier	Supprimer		15	9	Bonjour Yaacov, J'espère que tu bosses bien, je n...	23:08:05	1
Éditer	Copier	Supprimer		16	9	Bonjour Rephael, Je suis votre nouveau formateur.	23:08:35	4
Éditer	Copier	Supprimer		17	9	Hi Avi, Je prends la suite des cours...	23:09:12	5

II. ChatV1 – Fonctionnalités de base

L'objectif de cette première version était de valider le principe fondamental du chat : pouvoir envoyer un message et l'afficher à l'écran depuis la base de données.

Le chat étant codé en procédural, l'essentiel est dans l'index.

1. On a ici, la connexion BDD avec *mysqli*, insertions des messages en base, la requête pour récupérer les messages de la table *t_messages* (A ce stade, il n'y a que la table t_messages) :

```php
<?php
// Connexion base de données
$id = mysqli_connect(hostname: "localhost", username: "root", password: "root", database: "chatozar");

// Vérification que l'utilisateur est bien passé par le formulaire
if(isset($_POST['bouton'])) {

    // Récupérer les données
    $pseudo = $_POST['pseudo'];
    $message = $_POST['message'];

    // Insérer les données (message + pseudo)
    $insert = "INSERT INTO messages (expediteur, message, date)
               VALUES ('$pseudo', '$message', now())";

    mysqli_query(mysql: $id, query: $insert);
}
```

2. Voici le code pour afficher dynamiquement les messages dans une boucle PHP :

```php
<div class="messages">
    <ul>
        <?php
        // Récupérer les messages depuis la base
        $select = "SELECT * FROM messages ORDER BY date DESC";
        $resultat = mysqli_query(mysql: $id, query: $select);

        // Boucler les résultats et former les données dans un tableau
        while($data = mysqli_fetch_assoc(result: $resultat)) {
            echo '<li class="mess">'.$data['date'].' : '
                 .$data['expediteur'].' : '.$data['message'].'</li>';
        }
        ?>
    </ul>
</div>
```

3. Code HTML du formulaire d'envoie des messages :

```html
<!-- Formulaire d'envoie -->
<div class="form">
    <form action="" method="post">
        <input type="text" name="pseudo" placeholder="Pseudo :" required>
        <input type="text" name="message" placeholder="Votre message :" required><br>
        <input type="submit" value="Envoyer" name="bouton">
    </form>
</div>
```

4. Interface utilisateur simple :

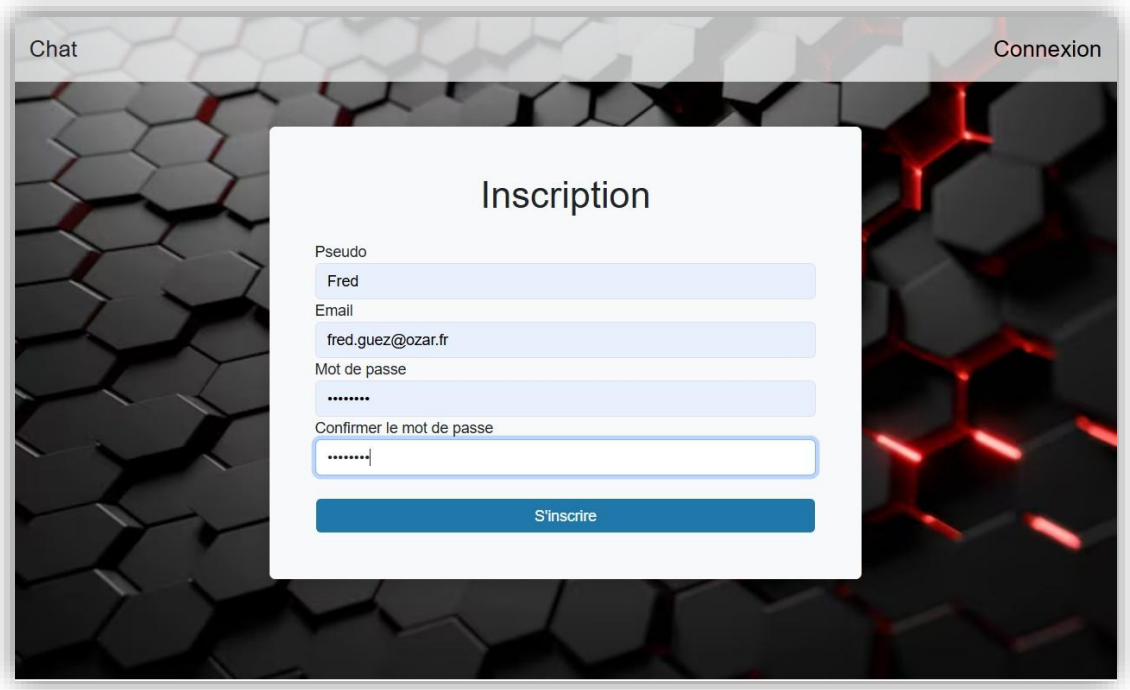
III. ChatV2 – Gestion des utilisateurs et amélioration de l'interface

La V2 apporte deux évolutions majeures par rapport à la V1 : l'ajout d'un système d'inscription et de connexion, et une refonte visuelle de l'interface avec Bootstrap et CSS personnalisé.

1. Formulaire d'inscription HTML + condition PHP pour l'erreur 'email existant' :

```html
<!-- Formulaire d'inscription -->
<form class="col-md-8 rounded form shadow p-5 mx-auto bg-light" action="" method="post">
    <h1 class="text-center mb-4">Inscription</h1>
    <!-- Si l'email existe déjà, renvoyer l'erreur prévue -->
    <?php if (isset($error_email)) { ?>
        <p class="alert alert-danger mt-3"> <?= $error_email ?></p>
    <?php } ?>
    <label for="">Pseudo</label>
    <input class="form-control" type="text" name="pseudo" id="pseudo"
        placeholder="Entrer le pseudo" required>
    <label for="">Email</label>
    <input class="form-control" type="text" name="email" id="email"
        placeholder="Entrer le email" required>
    <label for="">Mot de passe</label>
    <input class="form-control" type="password" name="password" id="password"
        placeholder="Entrer le mot de passe" required>
    <label for="">Confirmer le mot de passe</label>
    <input class="form-control" type="password" name="password2" id="password2"
        placeholder="Confirmer le mot de passe" required>
    <label for=""></label>
    <input class="form-control btn text-white" type="submit" name="inscrire" value="S'inscrire">
</form>
```

2. Interface amélioré utilisateur d'inscription dans le navigateur :



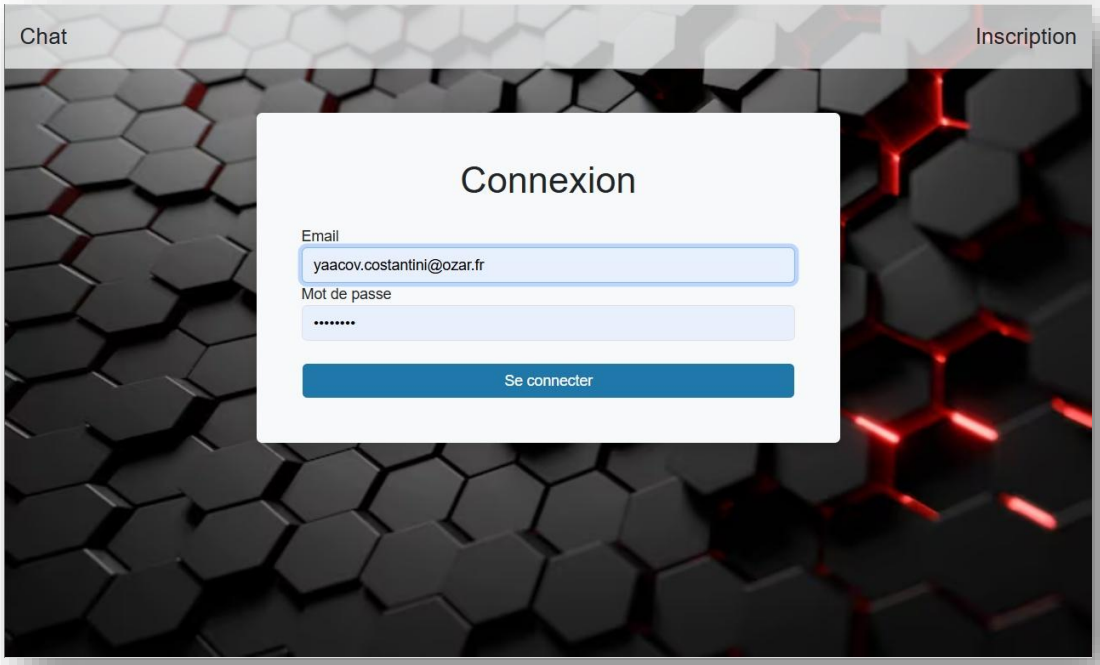
3. Traitement du formulaire d'inscription et gestion erreur :

```php
<?php
// Vérification que l'utilisateur a bien saisi ses informations
if (isset($_POST["inscrire"])) {
    // Mattre en variable les données récupérées via la superglobal $_POST
    $pseudo = $_POST['pseudo'];
    $email = $_POST['email'];
    $password = $_POST['password'];

    // Inclure le fichier contenant la connexion BDD
    include ('connect.php');

    // Comparer l'email saisi avec ceux de la table t_users
    $sql = "SELECT * FROM t_users WHERE email = '$email'";
    $result = mysqli_query(mysql: $id, query: $sql);

    // S'il existe un email identique renvoyer une erreur
    if (mysqli_num_rows(result: $result) > 0) { // Nombre de ligne supérieur à zéro
        $error_email = "Email déjà utilisé.";
    } else {
        // Sinon, insérer les informations saisies pour créer le nouvel utilisateur
        $add_user = "INSERT INTO t_users (pseudo, email, password)
        VALUES ('$pseudo', '$email', '$password')";
        mysqli_query(mysql: $id, query: $add_user);
        // Rediriger vers le login
        header(header: "Location: login.php");
        exit;
    }
```

4. Formulaire de connexion HTML :

```html
<!-- Formulaire de connexion -->
<form class="col-md-8 rounded form shadow p-5 mx-auto bg-light" action="" method="post">
    <h1 class="text-center mb-4">Connexion</h1>
    <?php if (isset($error)) { ?>
    <p class="alert alert-danger mt-3"> <?= $error ?></p>
    <?php } ?>
    <label for="">Email</label>
    <input class="form-control" type="text" name="email" id="email"
        placeholder="Entrer le email" required>
    <label for="">Mot de passe</label>
    <input class="form-control mb-4" type="password" name="password" id="password"
        placeholder="Entrer le mot de passe" required>
    <input class="form-control btn text-white" type="submit" name="connexion" value="Se connecter">
</form>
```

5. Interface de connexion (Navigateur) :



6. Traitement de connexion et stockage des données en session avec $_SESSION :

```php
<?php
// Démarrer la session
session_start();

// Vérifier que l'utilisateur a bien saisi les entrées
if (isset($_POST["connexion"])) {

    // Mettre en variable les données reçu par $_POST
    $email = $_POST['email'];
    $password = $_POST['password'];

    // Connexion BDD
    include ('connect.php');

    // Sélectionner, dans la table t_users, une ligne correspondante à l'email et
    // au MdP saisi par l'utilisateur
    $sql = "SELECT * FROM t_users WHERE email = '$email' AND password = '$password'";
    $result = mysqli_query(mysql: $id, query: $sql);

    // Sila requête ne renvoie rien (zéro), créer une erreur
    if (mysqli_num_rows(result: $result) == 0) {
        $error = "Email ou mot de passe invalide.";
    } else {
        // Sinon, stocker les informations nécessaires dans les variables de session
        $ligne = mysqli_fetch_assoc(result: $result);
        $_SESSION['id_user'] = $ligne['id'];
        $_SESSION['pseudo'] = $ligne['pseudo'];
        $_SESSION['email'] = $ligne['email'];
        $_SESSION['role'] = $ligne['role'];
        // Puis, rediriger l'utilisateur vers le chat      You, maintenant • Uncommitted c
        header(header: "Location: index.php");
        exit;
    }
}
```

7. Changements dans l'index :

   a. Gestion de la session et redirection si l'utilisateur n'est pas connecté :

```php
<?php
// Démmarrage de la session
session_start();

// Si l'utilisateur n'est pas connecté
if (!isset($_SESSION['pseudo'])) {
    // Renvoyer vers la connexion
    header(header: "Location: login.php");
    exit;
}
```

   b. Récupérer les utilisateurs qui serviront à la sélection des contacts pour l'affichage dynamique des messages d'une part, et pour l'envoie d'un message de l'autre :

```php
// Connexion BDD
include ('connect.php');

// Récupérer les utilisateurs ordonnés par ordre alphabétique
$contacts = "SELECT * FROM t_users ORDER BY pseudo";
$users = []; // $users sera un tableau
$users = mysqli_query(mysql: $id, query: $contacts);
```

   c. Sécurisation des injection SQL avec la requête préparée pour l'insertion des messages :

```php
// Insertion du message
if (isset($_POST['envoyer'])) {
    $expediteur = $_SESSION['id_user'];
    $message = $_POST['message'];
    $destinataire = $_POST['destinataire'];
    // Sécurisation avec la fonction prepare() pour protéger des injections SQL
    $stmt = $id->prepare(query: "INSERT INTO t_messages (expediteur, message, date, destinataire)
    VALUES (?, ?, now(), ?)");
    // Utilisation de bind_param pour paramétrer la requête
    $stmt->bind_param(types: "isi", var: &$expediteur, vars: &$message, $destinataire);
    var_dump(value: $stmt);
    $stmt->execute();
    header(header: "Location: index.php");
    exit();
}
```

   d. Récupération des messages en fonction de l'utilisateur :

```php
// Si les variables de session existent, l'utilisateur est donc connecté
if (isset($_SESSION)) {
    $id_user = $_SESSION['id_user'];
    // Récupérer les messages concernant l'utilisateur seulement
    $get_message = "SELECT m.*, u.pseudo AS nom_expe
                    FROM t_messages m
                    INNER JOIN t_users u ON m.expediteur = u.id
                    WHERE destinataire = '$id_user'
                    OR expediteur = '$id_user'";
    $posts = [];
    $posts = mysqli_query(mysql: $id, query: $get_message);
}
```

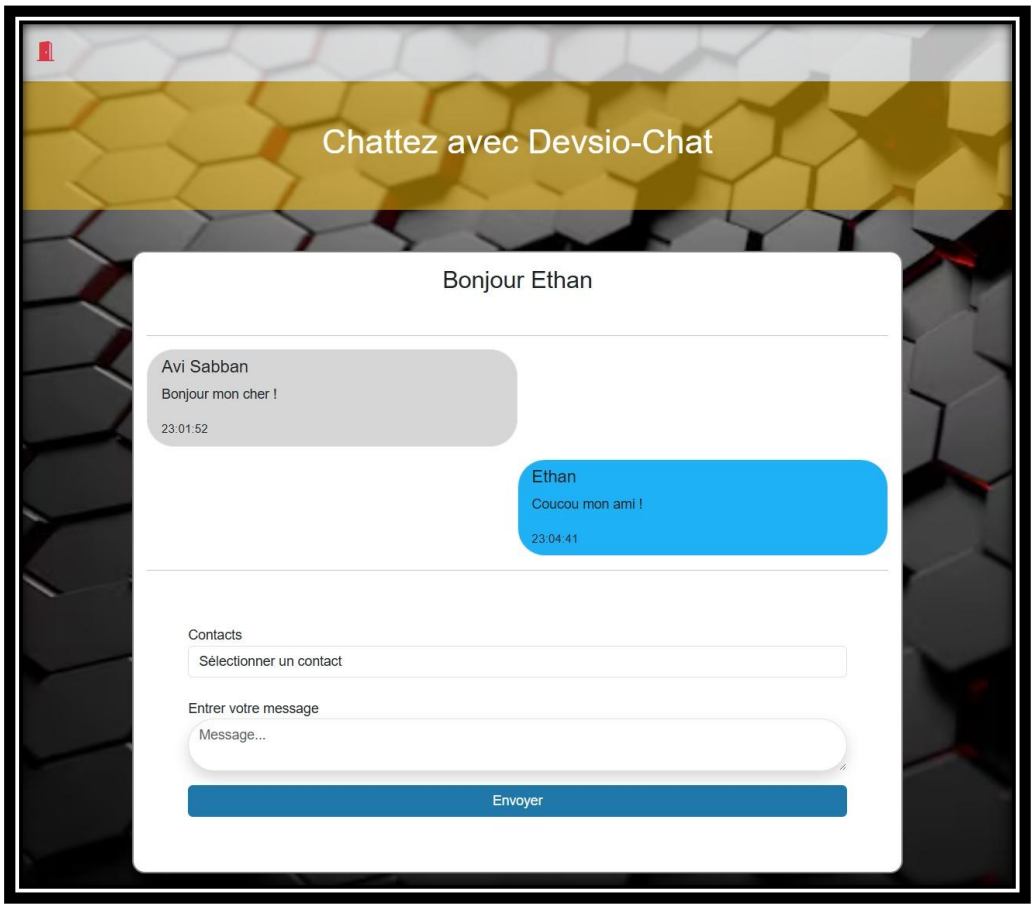
e. La classe CSS change en fonction de l'expéditeur et du destinataire : si l'expéditeur est l'utilisateur lui-même, les messages apparaitront à droite d'une couleur A, sinon à gauche d'une couleur B :

```
<ul class="list-group gap-3">
    <?php
        foreach ($posts as $post) {
            if ($post['nom_expe'] === $_SESSION['pseudo']) {
                $float = "ms-auto bg-self";
            } else {
                $float = "bg-foreign";
            }
    ?>

    <li class="list-group-item text-white rounded-5 w-50 <?= $float ?>">
        <h5><?= $post['nom_expe'] ?></h5>
        <p><?= $post['message'] ?></p>
        <small><?= $post['date'] ?></small>
    </li>
    <?php } ?>
</ul>
```

f. Formulaire d'envoie des messages avec sélection du contact :

```
<form class="form-group" method="post" class="formContent">
    <label class="m-2" for="destinataire">Contacts</label><br>
    <select class="form-control rounded-4 shadow-sm" name="destinataire" id="destinataire">
        <option value="" class="text-gray">Sélectionner un contact</option>
        <?php foreach ($users as $dest) { ?>
        <option value="<?= $dest['id'] ?>"><?= $dest['pseudo'] ?></option>
        <?php } ?>
    </select><br>
    <!-- <label for="message">Entrer votre message</label><br> -->
    <div class="position-relative">
        <textarea class="form-control mb-3 shadow rounded-5" name="message" id=""
        placeholder="Message..." required></textarea>
        <button class="position-absolute top-50 end-0 translate-middle-y me-2 btn
        rounded-circle text-white bi bi-send" type="submit" name="envoyer" value="Envoyer">
    </div>
</form>
```

g. Interface chat v2 amélioré :



8. Déconnexion :

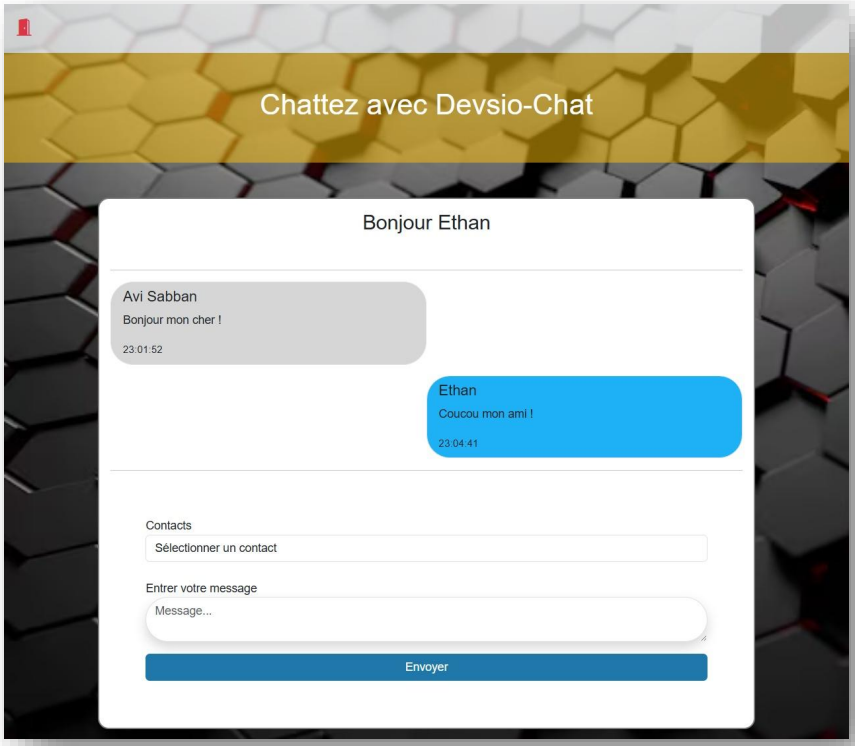
```php
<?php
session_start();
session_destroy();
header(header: "Location: login.php");
exit;
```

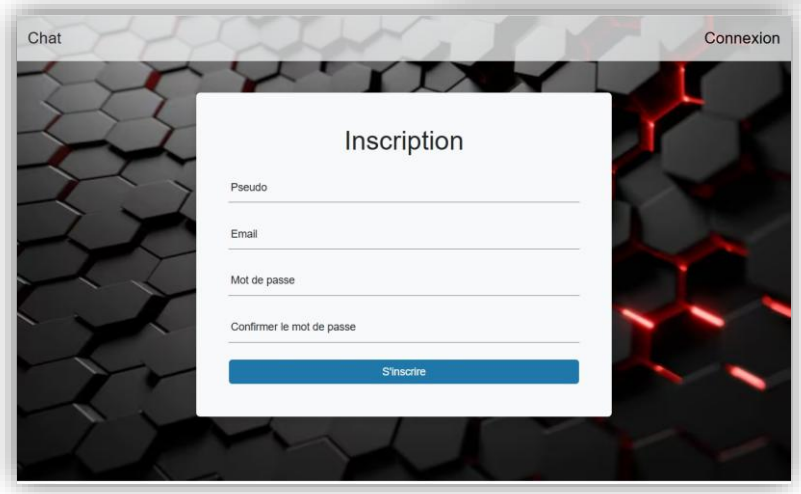
Interface navigateur version intermédiaire :

Nous avons commencé avec une version très simplifiée pour voir le rendu, puis dans un second temps, chacun a amélioré son affichage selon ses préférences. Celui-ci est ma version intermédiare.

Interface navigateur V2 :

Voici la version 2 du chat. Nous pouvons observer une amélioration au niveau du visuel mais aussi sur la répartition des messages en fonction de l'expéditeur et du destinataire





Inscription future :

Par la suite, l'application va subir une migration MVC, et ceci est une amélioration graphique de l'interface d'inscription.

CONCLUSION

Ce développement itératif en deux versions m'a permis de construire progressivement une application web dynamique complète en PHP procédural. La V1 a consolidé les bases : interaction avec une base de données MySQL, formulaires PHP, affichage dynamique. La V2 a introduit des notions essentielles comme la gestion de sessions, la navigation multipages et l'intégration d'un Framework CSS. Cette progression par étapes m'a aidé à mieux comprendre les limites du code procédural non structuré, ce qui a naturellement justifié la migration vers une architecture MVC en V3.

Passage en mode MVC
Une migration MVC n’est pas si simple mais dans ce cas, où le chat était relativement simple ça n’a pas été trop compliqué, et elle fera ainsi l’objet d’une autre activité.

EVOLUTION POSSIBLE

Le chat actuel permet d’inscrire un utilisateur, de le connecter, d’afficher les messages, et d’en envoyer. Cependant, l’amélioration principale pourrait consister à faire des groupes et de gérer les permissions et autorisations des membres.